

DOCUMENTACIÓN TÉCNICA

Sistema de Gestión Cafébar

Migración de Aplicación de Escritorio a Aplicación Web

Backend: Node.js + Express + Supabase (PostgreSQL)

Frontend: React + Vite + Tailwind CSS

Mateo Alejandro Merino Vida

5 de agosto de 2026

Tabla de Contenidos

Tabla de Contenidos	2
1. Introducción.....	4
1.1 Contexto del proyecto	4
1.2 Objetivo de la migración.....	4
1.3 Alcance	4
2. Arquitectura General	4
2.1 Stack tecnológico.....	4
2.2 Estructura del monorepo.....	5
2.3 Comunicación entre capas	5
3. Backend.....	5
3.1 Estructura de carpetas.....	5
3.2 Patrón arquitectónico	5
3.3 Autenticación y autorización	6
3.4 Módulos del backend	6
3.5 Flujo completo de Caja (POS).....	7
3.6 Reglas de negocio implementadas	7
3.7 Optimización de consultas	7
4. Frontend.....	8
4.1 Estructura de carpetas (Atomic Design)	8
4.2 Patrones de diseño del frontend	8
4.3 Módulos del frontend (carpeta features/)... ..	8
4.4 Control de acceso por rutas	9
4.5 Manejo de sesión expirada.....	9
5. Base de Datos	9
5.1 Motor.....	9
5.2 Tablas principales	9
5.3 Integridad referencial	10
5.4 Scripts de base de datos	10
6. Funcionalidades Destacadas	11
6.1 Impresión de tickets y comandas.....	11
6.2 Personalización por unidad	11
6.3 Backup de la base de datos	11
6.4 Eliminación física de registros.....	11
6.5 Reportes.....	11
6.6 Comunicaciones por correo	11
7. Seguridad	12
8. Despliegue	12
8.1 Estado actual.....	12
8.2 Plan de migración a alojamiento local	12

8.3 Variables de entorno requeridas	12
--	-----------

1. Introducción

1.1 Contexto del proyecto

El sistema Cafebar era originalmente una aplicación de escritorio desarrollada en Python con la biblioteca PyQt y una base de datos SQLite local, instalada en una única computadora del restaurante. Este documento describe la migración completa de dicho sistema hacia una aplicación web moderna, manteniendo y adaptando toda la lógica de negocio original, corrigiendo inconsistencias detectadas en el sistema original, y aprovechando las ventajas de una arquitectura cliente-servidor.

El sistema es de uso exclusivamente interno para la gestión del restaurante (registro de pedidos, cocina, cobro, administración de productos, empleados y reportes). No incluye funcionalidades de pedidos a domicilio ni tienda en línea para clientes finales.

1.2 Objetivo de la migración

Trasladar la funcionalidad completa del sistema de escritorio a una arquitectura web (backend + frontend + base de datos en la nube), permitiendo el acceso simultáneo desde múltiples dispositivos dentro del local (Caja, cocina, administración), sin depender de una única computadora física para operar.

1.3 Alcance

Se migraron y adaptaron todos los módulos funcionales del sistema original: autenticación, gestión de empleados y cargos, categorías y productos, stock de ingredientes, secciones y mesas, promociones, catálogo Familia, Caja (POS) completo, Configuración (incluyendo backup y eliminación física de registros) y Reportes.

Quedó fuera del alcance de esta primera etapa el módulo de Clientes del sistema original, ya que se confirmó que no estaba conectado al flujo real de ventas en ningún punto del sistema original (módulo desconectado del resto de la aplicación).

2. Arquitectura General

2.1 Stack tecnológico

Capa	Tecnología
Backend	Node.js + Express
Base de datos	PostgreSQL (vía Supabase)
Frontend	React + Vite + Tailwind CSS v4
Autenticación	JWT (8 horas de duración) + bcrypt
Correo transaccional	Brevo (API REST)
Almacenamiento de imágenes	Supabase Storage
Gráficos (Reportes)	Recharts

Capa	Tecnología
Generación de Excel	librería xlsx
Envío de correo con adjuntos	nodemailer / API Brevo

2.2 Estructura del monorepo

El proyecto está organizado como un monorepo con dos aplicaciones independientes, cada una con su propio gestor de dependencias, más una carpeta adicional con la definición de la base de datos:

```
CAFE_BAR_WEB/
├── backend/    (API REST en Node.js + Express)
├── frontend/   (Aplicación React servida con Vite)
└── database/   (Scripts SQL: schema, índices, triggers, permisos, datos)
```

2.3 Comunicación entre capas

El frontend se comunica con el backend exclusivamente a través de peticiones HTTP (API REST) autenticadas con un token JWT. El backend es el único componente que se comunica directamente con la base de datos, utilizando la clave de servicio (service_role) de Supabase, la cual omite las políticas de seguridad a nivel de fila (RLS) y centraliza toda la lógica de permisos en el propio backend.

```
Navegador (React)
  | HTTPS + Bearer Token (JWT)
  ▼
Backend (Node/Express)
  | Supabase JS Client (service_role)
  ▼
Base de datos (PostgreSQL / Supabase)
```

3. Backend

3.1 Estructura de carpetas

```
backend/src/
├── config/      Conexión a Supabase, configuración de subida de archivos (multer)
├── controllers/ Reciben la petición, validan la entrada, llaman al service, responden
├── services/    Lógica de negocio y consultas a Supabase
├── routes/      Definición de endpoints (URL → controller)
├── middlewares/ authenticate (verifica JWT) y authorize (verifica permiso)
├── utils/       Funciones puras reutilizables (hash, tokens, validadores)
└── scripts/     Scripts puntuales (ej. creación del usuario DIRECTORIO)
```

3.2 Patrón arquitectónico

Cada funcionalidad del backend sigue estrictamente la cadena Route → Controller → Service:

- **Route:** define el método HTTP, la URL, el middleware de autenticación y el permiso requerido, y delega en el controlador correspondiente.
- **Controller:** valida los datos de entrada (campos obligatorios, formatos), invoca al servicio y traduce el resultado (o el error) a una respuesta HTTP con el código de estado adecuado.
- **Service:** contiene toda la lógica de negocio real y las consultas a Supabase. Es la única capa que conoce la estructura de las tablas de la base de datos.

3.3 Autenticación y autorización

El sistema utiliza JSON Web Tokens (JWT) con una duración de 8 horas. Existen dos tipos de sesión:

- **DIRECTORIO:** usuario administrador único, autenticado con una contraseña propia (almacenada con hash bcrypt en la tabla directorio), con acceso a todos los permisos del sistema de forma fija.
- **Empleado:** autenticado con su alias y contraseña. Sus permisos se determinan dinámicamente a partir del Cargo al que pertenece, combinando los permisos a nivel de pantalla (tabla permisos_cargo) y a nivel de acción específica dentro de una pantalla (tabla permisos_cargo_subpantalla).

Cada endpoint protegido utiliza dos middlewares en cadena: `authenticate` (verifica que el token JWT sea válido y no haya expirado) y `authorize('PERMISO')` (verifica que el usuario autenticado tenga el permiso específico requerido para esa acción).

3.4 Módulos del backend

Módulo	Endpoints base	Responsabilidad
Auth	/api/auth	Login de empleados y DIRECTORIO, emisión de JWT
Password Reset	/api/password-reset	Recuperación de contraseña por código enviado a correo (Brevo)
Screens	/api/screens	Catálogo de pantallas y subpantallas para permisos
Roles (Cargos)	/api/roles	CRUD de cargos y asignación de permisos por pantalla/subpantalla
Employees	/api/employees	CRUD de empleados, dar de baja, reseteo de contraseña
Categories	/api/categories, /api/subcategories	CRUD de categorías y subcategorías de productos
Products	/api/products	CRUD de productos, ingredientes y recetas, imágenes
Stock	/api/stock	CRUD de ingredientes y cantidades disponibles
Sections	/api/sections	Administración de secciones (salones) y mesas
Promotions	/api/promotions	CRUD de promociones, horarios de disponibilidad, ingredientes por producto
POS (Caja)	/api/pos	Flujo completo de venta: mesas, carrito, cocina, cobro, impresión
Config	/api/config	Enlace de impuestos, contraseña DIRECTORIO, ventas diarias
Backup	/api/backup	Exportar/importar base de datos en Excel, envío por correo
Deletion	/api/deletion	Eliminación física en cascada de empleados, cargos, secciones y ventas
Reports	/api/reports	Reportes de ventas detalladas, resumen por fechas, productos y empleados

3.5 Flujo completo de Caja (POS)

El módulo de Caja es el núcleo operativo del sistema. Su flujo completo es el siguiente:

- 1. Selección de sección y mesa (o continuación de una mesa ya ocupada).
- 2. Armado del pedido: búsqueda de productos por categoría/subcategoría o de promociones vigentes, con personalización de ingredientes (exclusiones y extras) por unidad.
- 3. Registro de la venta: se descuenta el stock correspondiente y se generan los registros de detalle por producto/promoción.
- 4. Envío a cocina: impresión de la comanda (solo los productos nuevos del envío actual, con las promociones desglosadas en sus productos individuales).
- 5. Marcado de unidades listas: pantalla de cocina que agrupa por producto y personalización, permitiendo marcar de a una o todas las unidades pendientes.
- 6. Cobro: validación de que no queden unidades sin marcar, selección de método de pago (efectivo, QR o mixto), impresión del ticket para el cliente (con las promociones agrupadas, sin desglosar) y liberación de la mesa.

3.6 Reglas de negocio implementadas

- Una venta tiene solo 2 estados reales: PENDIENTE (en preparación) o Finalizado. El cobro finaliza todos los detalles de una venta a la vez, de forma atómica — no existen estados parciales ni intermedios.
- El stock nunca bloquea el registro de una venta. Si la cantidad disponible no alcanza, se descuenta hasta 0 (nunca queda negativo), pero la venta se registra igual, evitando que un stock desactualizado le cueste una venta real al negocio.
- No se puede cobrar una mesa mientras existan unidades de productos sin marcar como listas en cocina.
- Control de impresión: cada venta permite un máximo de 2 impresiones gratuitas (ticket y comanda de cocina se cuentan por separado). A partir de la tercera, el sistema exige la contraseña del usuario actual antes de continuar.
- El número de venta se reinicia cada día, calculado en huso horario de Bolivia (UTC-4 fijo), sin depender de la configuración horaria del servidor donde corra el backend.
- Personalización de productos y promociones: excluir un ingrediente nunca reduce el precio del producto; agregar un ingrediente extra sí se suma al precio base. Una misma cantidad de un producto puede dividirse en sub-grupos con personalización distinta cada uno (ejemplo: de 2 hamburguesas, 1 sin cebolla y 1 sin personalizar).
- Las promociones tienen horarios de disponibilidad configurables (fecha específica, rango de fechas, días de la semana, rango horario) y solo se muestran en Caja cuando están vigentes en el momento exacto de la consulta.

3.7 Optimización de consultas

Durante el desarrollo se identificaron y corrigieron múltiples casos de patrón N+1 (una consulta adicional por cada elemento de una lista), reemplazándolos por consultas masivas con el operador `.in()` de Supabase seguidas de un agrupamiento en memoria mediante estructuras Map de JavaScript. Este patrón se aplicó de forma consistente en: la pantalla de marcar pendientes en cocina, el reporte de ventas detalladas con su desglose de personalización, y el cálculo del árbol de dependencias en la eliminación física de registros.

4. Frontend

4.1 Estructura de carpetas (Atomic Design)

```
frontend/src/
├── components/
│   ├── atoms/      Piezas mínimas: Button, TextInput, Modal, Checkbox, Toast...
│   ├── molecules/  Combinaciones simples: FormField, SearchBar, Pagination...
│   ├── organisms/  Piezas complejas con lógica propia: modales, tablas, carritos
│   └── templates/  Layout general de página (MainLayout)
├── features/       Un folder por módulo de negocio (pages, hooks, services, utils)
├── context/        AuthContext (sesión y permisos)
├── hooks/          Hooks genéricos compartidos (usePagination, useDisclosure...)
├── router/         AppRouter + ProtectedRoute
├── constants/      Colores, navegación
└── lib/            Cliente axios con interceptores de autenticación
```

4.2 Patrones de diseño del frontend

- Atomic Design en el frontend: los componentes se organizan en atoms, molecules, organisms y templates. Toda la lógica de estado vive en hooks personalizados dentro de cada módulo (features/<módulo>/hooks); los componentes solo reciben datos y funciones ya resueltas.
- Patrón Router → Controller → Service en el backend: las rutas solo mapean URLs a controladores; los controladores validan la entrada y dan forma a la respuesta HTTP; toda la lógica de negocio y las consultas a Supabase viven exclusivamente en los servicios.
- Prevención de consultas N+1: cuando una pantalla necesita datos relacionados de muchos registros a la vez (por ejemplo, las exclusiones y extras de cada producto de una venta), el backend trae todo en un pequeño número de consultas con el operador .in(...) y arma las relaciones en memoria usando estructuras Map, en vez de una consulta por cada registro.
- Rutas protegidas por permiso: cada endpoint del backend usa el middleware authorize('NOMBRE_PERMISO'), y cada ruta del frontend está envuelta en <ProtectedRoute permission="NOMBRE_PERMISO">. Los permisos de un empleado provienen de su Cargo (tablas permisos_cargo y permisos_cargo_subpantalla); el usuario DIRECTORIO tiene una lista fija de permisos definida en el backend.
- Componentes de UI reutilizados de forma consistente en todo el sistema: SearchableSelect en vez de <select> nativo, usePagination + Pagination para toda tabla o listado, y Toast para mensajes de confirmación/error, siempre renderizado como hermano del layout principal (nunca anidado dentro de él, por una restricción de posicionamiento CSS del layout).

4.3 Módulos del frontend (carpeta features/)

Carpeta (features/)	Contenido
auth	Login, recuperación de contraseña
roles	Registro y modificación de Cargos
employees	Registro, modificación y baja de Empleados
categories	Registro, modificación y baja de Categorías/Subcategorías
products	Registro, modificación y baja de Productos
stock	Registro y modificación de Ingredientes
sections	Administración de Secciones y Mesas

Carpeta (features/)	Contenido
promotions	Registro, modificación y baja de Promociones
catalog	Familia: catálogo explorador de productos/promociones
pos	Caja: todo el flujo de venta (carrito, cocina, cobro, impresión)
config	Configuración: enlace de impuestos, contraseña, ventas diarias, backup, eliminación física
reports	Reportes: ventas detalladas, resumen por fechas, productos, gráficos por empleado

4.4 Control de acceso por rutas

Cada ruta de la aplicación está protegida mediante el componente `<ProtectedRoute permission="NOMBRE_PERMISO">`, que verifica en el `AuthContext` si el usuario autenticado posee el permiso indicado antes de renderizar la página; en caso contrario, redirige automáticamente. El menú de navegación (`NavMenu`) también filtra dinámicamente las opciones visibles según los permisos del usuario.

4.5 Manejo de sesión expirada

Se implementó un interceptor global de respuestas HTTP (en el cliente axios) que detecta cualquier respuesta con código 401 (token inválido o expirado) y dispara un evento del navegador. Un componente montado en la raíz de la aplicación escucha ese evento y muestra un aviso modal explicando que la sesión expiró, cerrando la sesión local y redirigiendo al usuario a la pantalla de inicio de sesión.

5. Base de Datos

5.1 Motor

PostgreSQL, administrado a través de Supabase, que provee además autenticación propia (no utilizada en este proyecto, ya que se implementó autenticación JWT propia por sus requisitos particulares), almacenamiento de archivos (Storage) y una API REST autogenerada (PostgREST) consumida mediante el cliente oficial supabase-js.

5.2 Tablas principales

Tabla	Propósito
directorio	Usuario administrador único del sistema
cargo / permisos_cargo / permisos_cargo_subpantalla	Roles del personal y sus permisos
empleado	Personal del restaurante
categoria / subcategoria	Clasificación de productos
producto / productos_ingredientes	Catálogo de productos y sus recetas

Tabla	Propósito
stock	Ingredientes y cantidades disponibles
seccion / mesa	Salones y mesas del restaurante
promocion / promocion_prod / promocion_dias	Combos promocionales y su disponibilidad
venta / detalles_venta / detalles_venta_unidades	Ventas registradas y su detalle por unidad
detalles_venta_exclusiones(_promo) / detalles_venta_extras	Personalización de ingredientes por venta
pago / metodo_pago	Pagos registrados por venta
cuenta	Cuentas asociadas para funciones de correo
enlace	Enlace configurable al portal de impuestos (SIAT)

5.3 Integridad referencial

Las relaciones entre tablas utilizan reglas explícitas de eliminación (ON DELETE RESTRICT, CASCADE o SET NULL según el caso) definidas directamente en el esquema de PostgreSQL. Para operaciones de eliminación en cascada controladas desde la aplicación (por ejemplo, la eliminación física de un empleado y todas sus ventas asociadas), el backend realiza el borrado en el orden correcto —de las tablas dependientes hacia las tablas principales— replicando de forma explícita la lógica de dependencias, en vez de depender únicamente del comportamiento automático de las restricciones de la base de datos.

5.4 Scripts de base de datos

La definición completa de la base de datos vive en la carpeta database/, en la raíz del repositorio, separada de backend/ y frontend/ por no ser código de ninguna de las dos aplicaciones. Contiene 6 scripts que deben ejecutarse en un orden específico (no alfabético) para reconstruir la base desde cero, junto con un README propio que documenta ese orden:

Orden	Archivo	Qué hace
1	schema.sql	Crea todas las tablas del sistema
2	index.sql	Crea los índices para búsquedas y filtros rápidos
3	triggers.sql	Funciones automáticas (numeración de mesas, conteo de mesas por sección)
4	permits.sql	Activa la seguridad a nivel de fila (RLS), bloqueando el acceso directo a la base
5	data population.sql	Datos iniciales para que el sistema arranque, más datos de prueba
6	functions backups.sql	Funciones que usa Backup para exportar/importar la base en Excel

El usuario administrador (DIRECTORIO) no se crea con estos scripts, ya que requiere una contraseña ya encriptada; se genera aparte ejecutando una sola vez `backend/src/scripts/seedDirectorio.js`.

6. Funcionalidades Destacadas

6.1 Impresión de tickets y comandas

El sistema no requiere configurar ninguna impresora dentro de la aplicación ni en la base de datos. El ticket de venta y la comanda de cocina se generan como texto plano en formato monoespaciado (ancho de 44 caracteres, replicando el formato de una impresora térmica de 80mm), y se imprimen abriendo una ventana nueva del navegador que invoca el diálogo de impresión nativo del sistema operativo, donde el usuario selecciona la impresora física instalada.

6.2 Personalización por unidad

Tanto los productos individuales como los productos que forman parte de una promoción permiten personalización a nivel de unidad individual: es posible dividir una misma cantidad solicitada en distintos sub-grupos, cada uno con su propia combinación de ingredientes excluidos y/o extras agregados, reflejándose correctamente tanto en el precio final como en la comanda de cocina.

6.3 Backup de la base de datos

Desde el módulo de Configuración es posible exportar el contenido completo de la base de datos a un archivo Excel (una hoja por tabla), importar un archivo Excel para restaurar el estado completo de la base de datos (operación protegida con una confirmación explícita de texto), y enviar el archivo de backup por correo electrónico utilizando el servicio Brevo ya configurado para el resto de las comunicaciones del sistema.

6.4 Eliminación física de registros

Herramienta administrativa que permite el borrado permanente (no reversible) de Empleados, Cargos, Secciones y Ventas. Antes de ejecutar cualquier eliminación, el sistema calcula y muestra al usuario un árbol de dependencias con todos los registros relacionados que también serán eliminados en cascada (por ejemplo, al eliminar un Cargo se muestran los empleados asociados y, de estos, sus ventas), exigiendo una confirmación explícita.

6.5 Reportes

El módulo de Reportes permite generar cuatro tipos de análisis sobre un rango de fechas: ventas detalladas (con el desglose completo de productos, personalización y el mesero responsable de cada línea del pedido, no solo quien abrió la mesa), resumen por fechas agrupado por cajero, productos más vendidos, y un resumen visual por empleado mediante gráficos de torta. En el gráfico "Ventas por Mesero", cada mesero que participó en una venta (aunque no la haya abierto) suma una participación, a diferencia de "Ventas por Cajero" donde cada venta tiene un único responsable de cobro.

Cualquier reporte generado puede imprimirse/guardarse como PDF a través del navegador, o enviarse por correo electrónico como un archivo PDF adjunto real (no como texto dentro del cuerpo del correo). Ese PDF se genera en el backend con Puppeteer, que renderiza la misma plantilla HTML del reporte en un navegador Chromium sin interfaz y la exporta a PDF, preservando el diseño exacto de tablas y totales.

6.6 Comunicaciones por correo

Todos los correos que envía el sistema (recuperación de contraseña, backups y reportes) se generan a través de Brevo con una plantilla HTML de marca consistente: encabezado con degradado azul, el

nombre del sistema, y el contenido específico de cada caso (código de verificación, aviso de adjunto, etc.), en vez de texto plano sin estilo.

7. Seguridad

- Las contraseñas de empleados y del usuario DIRECTORIO se almacenan siempre con hash bcrypt, nunca en texto plano.
- La sesión se gestiona mediante JWT firmado con una clave secreta del servidor, con expiración de 8 horas.
- Cada endpoint del backend valida explícitamente el permiso requerido antes de ejecutar cualquier acción, independientemente de las restricciones del frontend.
- Las credenciales sensibles (claves de Supabase, secreto JWT, credenciales de Brevo) se gestionan exclusivamente mediante variables de entorno (.env), excluidas del control de versiones.
- El acceso a la base de datos desde el backend utiliza la clave de servicio de Supabase (service_role), por lo que toda la responsabilidad de control de acceso recae en la capa de autorización del backend, no en políticas de base de datos.

8. Despliegue

8.1 Estado actual

El sistema se encuentra actualmente desplegado utilizando Supabase Cloud como base de datos, con el backend y el frontend ejecutándose de forma independiente y comunicándose vía HTTPS.

8.2 Plan de migración a alojamiento local

Está prevista una migración hacia un esquema de auto-alojamiento (Supabase self-hosted mediante Docker), ejecutado en la misma computadora utilizada como punto de Caja del restaurante. Este cambio no requiere modificaciones al código de la aplicación, únicamente el cambio de la URL de conexión a la base de datos, y tiene como principal beneficio que el sistema continúe operando con normalidad ante una interrupción del servicio de internet del local, manteniendo el acceso simultáneo desde múltiples dispositivos dentro de la misma red local.

8.3 Variables de entorno requeridas

Backend:

```
SUPABASE_URL
SUPABASE_SERVICE_ROLE_KEY
JWT_SECRET
BREVO_API_KEY
BREVO_SENDER_EMAIL
BREVO_SENDER_NAME
```

Frontend:

```
VITE_API_URL
```